MATERIA

INF761

Desarrollo de Videojuegos

Guía de Laboratorio 2

"Brick Breaker": Física 2D y Collisiones

Carrera: Ingeniería Informática

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Semestre: I-2026

Universidad: Universidad Autónoma Tomás Frías (UATF)

1. Objetivos de Aprendizaje

- 1. Comprender el funcionamiento del motor de física 2D de Unity (PhysX 2D) y sus componentes principales: Rigidbody2D, Collider2D y PhysicsMaterial2D.
- 2. Diferenciar los tres tipos de Rigidbody2D (Dynamic, Kinematic, Static) y aplicar cada uno al elemento correcto del juego.
- 3. Crear y configurar un PhysicsMaterial2D para simular rebote perfecto sin pérdida de energía.
- 4. Implementar el movimiento de la paleta mediante Kinematic Rigidbody2D y el método MovePosition.
- 5. Programar la detección de colisiones físicas con OnCollisionEnter2D para destruir bloques dinámicamente.
- 6. Distinguir conceptualmente entre una Colisión física (OnCollisionEnter2D) y un Trigger (OnTriggerEnter2D), eligiendo el mecanismo correcto para cada situación.
- 7. Construir un sistema de puntuación con UI TextMeshPro y el patrón de diseño Singleton.
- 8. Programar una condición de derrota cuando la pelota cae por debajo de la paleta.
- 9. Optimizar la detección de colisiones mediante la configuración de la Layer Collision Matrix.

2. Requisitos Previos y Materiales

2.1 Requisitos de Software

Herramienta	Requisito
Unity Hub	Versión 3.x o superior
Unity Editor	2022 LTS o 2023 LTS (misma versión usada en la Guía 1)
Visual Studio / VS Code	Con soporte para C# y Unity Tools
Guía 1 completada	Proyecto CatchTheFruit funcionando, o proyecto nuevo vacío 2D

2.2 Requisitos de Conocimiento

- Crear GameObjects en Unity y configurar sus componentes desde el Inspector.
- Crear scripts C# (MonoBehaviour) y adjuntarlos a GameObjects.
- Estructura básica de un script Unity: Awake, Start, Update.
- Concepto de Prefab y cómo instanciarlo con Instantiate().

> **Nota importante**
> Si no completaste la Guía 1, puedes crear un proyecto Unity nuevo con la plantilla 2D y seguir esta guía de forma independiente. Todos los GameObjects se crean desde cero en los pasos de desarrollo.

3. Marco Teórico

3.1 El Motor de Física 2D de Unity (PhysX 2D)

Unity emplea el motor Box2D —expuesto a través de su capa PhysX 2D— para simular movimiento, gravedad y colisiones en escenas bidimensionales. A diferencia del sistema de física 3D, el motor 2D opera exclusivamente en el plano XY e ignora el eje Z. Tres componentes forman la base de este sistema: Rigidbody2D, Collider2D y PhysicsMaterial2D.

3.2 Rigidbody2D: Tipos y Propiedades

El componente Rigidbody2D le indica a Unity que un GameObject debe ser gobernado por el motor de física. Sin él, el objeto no responde a fuerzas ni a colisiones físicas. Existen tres tipos:

Tipo	Descripción
Dynamic	El cuerpo responde a fuerzas, gravedad y colisiones. Es el tipo estándar para objetos que se mueven de forma realista. En Brick Breaker: la pelota.
Kinematic	No responde a fuerzas externas. Solo se mueve mediante código (MovePosition o velocity). Ideal para la paleta controlada por el jugador.
Static	Nunca se mueve. Unity lo optimiza internamente. Adecuado para paredes y suelo del nivel.

Las propiedades más relevantes para este proyecto son:

- Mass: masa del cuerpo. Para la pelota se recomienda el valor 1.
- Gravity Scale: multiplicador de la gravedad global. Debe ser 0 tanto en la paleta como en la pelota (el juego no usa gravedad).
- Linear Drag: rozamiento con el "aire". Debe ser 0 para mantener la velocidad constante.
- Angular Drag: resistencia a la rotación. Mantener en 0.
- Collision Detection: usar Continuous en objetos rápidos (como la pelota) para evitar que atraviese paredes en un solo frame.
- Constraints: permite bloquear posición o rotación en ejes específicos. La paleta debe tener bloqueado Freeze Position Y y Freeze Rotation Z.

3.3 Collider2D: Tipos de Colisionadores

El Collider2D define la forma física de un objeto para la detección de colisiones. Es completamente independiente de la apariencia visual (SpriteRenderer). Los tipos más comunes son:

Collider	Descripción y uso en el proyecto
BoxCollider2D	Forma rectangular. Muy eficiente. Se usará en la paleta, los bloques y las paredes.
CircleCollider2D	Círculo perfecto. Produce rebotes más predecibles y uniformes. Se usará en la pelota.
EdgeCollider2D	Línea o polilínea sin relleno interior. Útil para los bordes de pantalla. No tiene volumen, lo que lo hace muy liviano.

PolygonCollider2D	Polígono arbitrario. Para objetos de forma irregular. No necesario en este proyecto.

3.4 PhysicsMaterial2D: Fricción y Bounciness

PhysicsMaterial2D es un asset que se asigna directamente a un Collider2D y define el comportamiento de la superficie al colisionar:

- Friction (fricción): resistencia al deslizamiento entre dos superficies. Rango de 0 a 1. Para la pelota: 0.
- Bounciness (elasticidad): energía conservada al rebotar. Rango de 0 a 1. Para la pelota: 1 (rebote perfecto).

> **Importante**
>
> Para lograr rebote perfecto sin pérdida de energía, ambas superficies en contacto deben tener Friction = 0 y Bounciness = 1, o bien el material debe asignarse solo a la pelota con esos valores.

3.5 Colisión vs. Trigger: Diferencia fundamental

Esta distinción es una de las más importantes en física de Unity. Ambos mecanismos detectan cuándo dos objetos se solapan, pero se comportan de manera opuesta:

Aspecto	Collision (IsTrigger = false)	Trigger (IsTrigger = true)
Bloqueo físico	SÍ — los cuerpos se detienen o rebotan al contacto.	NO — los objetos se atraviesan mutuamente.
Respuesta física	SÍ — respeta PhysicsMaterial2D (rebote, fricción).	NO — solo detección, sin respuesta física.
Método de detección	OnCollisionEnter2D(Collision2D col)	OnTriggerEnter2D(Collider2D other)
Parámetro recibido	Collision2D: contiene puntos de contacto, normal, Rigidbody impactante.	Collider2D: solo el collider del objeto que entró.
Uso en este proyecto	Pelota ↔ paleta, pelota ↔ bloques, pelota ↔ paredes.	Zona de muerte (DeathZone) debajo de la paleta.

> **Error común**
>
> Muchos estudiantes activan IsTrigger cuando esperan que la pelota rebote. Si IsTrigger está activo, la pelota atravesará el objeto sin rebotar. Usa IsTrigger = false (Collision) para rebotes, e IsTrigger = true (Trigger) únicamente para detección sin respuesta física, como la zona de derrota.

3.6 Layer Collision Matrix

Unity permite controlar qué capas (Layers) interactúan físicamente con qué otras capas, mediante la Layer Collision Matrix en Edit → Project Settings → Physics 2D. Desactivar pares de capas que no necesitan interactuar reduce el número de cálculos de colisión por frame, mejorando el rendimiento del juego.

Por ejemplo, los bloques (Brick) no necesitan detectar colisiones entre sí, ni con las paredes, ya que son estáticos e inmóviles. Únicamente la pelota (Ball) debe colisionar con los tres tipos de objetos.

4. Desarrollo Paso a Paso

Paso 1: Crear el proyecto Unity 2D

1 Nuevo proyecto	Abra Unity Hub → New Project. Seleccione la plantilla **2D Built-In Render Pipeline**. Nombre el proyecto BrickBreaker y elija una ubicación en su disco.
2 Abrir el proyecto	Haga clic en Create Project. Unity cargará el editor. La primera carga puede tomar varios minutos.
3 Renombrar escena	En la ventana Project, dentro de Assets/Scenes, haga doble clic sobre SampleScene. Renómbrela como GameScene (F2 o clic derecho → Rename).
4 Configurar cámara	Seleccione Main Camera en Hierarchy. En el Inspector verifique que Projection = Orthographic y Size = 5. Cambie el color de Background a un color oscuro (por ejemplo, negro o gris muy oscuro).

Paso 2: Crear el PhysicsMaterial2D para rebote perfecto

Antes de colocar objetos en la escena, cree el material de física que hará que la pelota rebote sin perder energía.

1 Crear material	En el panel Project, clic derecho sobre la carpeta Assets → Create → 2D → Physics Material 2D.
2 Nombrar	Asigne el nombre BallMaterial al archivo recién creado.
3 Configurar	Seleccione BallMaterial. En el Inspector establezca: Friction = 0 y Bounciness = 1.

Paso 3: Crear la paleta (Paddle) con Kinematic Rigidbody2D

1 Crear objeto	GameObject → 2D Object → Sprites → Square. En Hierarchy renombre el objeto a Paddle.
2 Posición y escala	En el componente Transform: Position (0, −4, 0) y Scale (2.5, 0.3, 1). En SpriteRenderer asigne un color a elección (por ejemplo, un morado).
3 BoxCollider2D	Add Component → Box Collider 2D. No modifique el tamaño; se ajusta automáticamente al sprite.
4 Rigidbody2D	Add Component → Rigidbody 2D. Configure: Body Type = Kinematic. En Constraints active Freeze Position Y y Freeze Rotation Z.
5 Capa (Layer)	En la parte superior del Inspector, despliegue Layer → Add Layer... Cree las capas: Ball, Paddle, Brick, Wall. Vuelva a seleccionar Paddle y asígnele la capa Paddle.

Paso 4: Crear la pelota (Ball) con Dynamic Rigidbody2D

1 Crear objeto	GameObject → 2D Object → Sprites → Circle. Renombre a Ball. Transform: Position (0, −3, 0), Scale (0.4, 0.4, 1).
2 CircleCollider2D	Add Component → Circle Collider 2D. En el campo Material de este collider, arrastre el asset BallMaterial creado en el Paso 2.
3 Rigidbody2D	Add Component → Rigidbody 2D. Configure: Body Type = Dynamic, Mass = 1, Gravity Scale = 0, Linear Damping = 0, Angular Damping = 0, Collision Detection = Continuous. En Constraints active Freeze Rotation Z.
4 Tag y capa	En el Inspector, asigne Tag = Ball (crear el tag si no existe) y Layer = Ball.

Paso 5: Crear las paredes y el límite superior

Las paredes delimitan el área de juego. No llevan Rigidbody2D porque son estáticos; Unity los trata automáticamente como Static Colliders.

1 Objeto contenedor	Cree un GameObject vacío (GameObject → Create Empty) y nómbrelo Walls. Este objeto solo sirve de agrupador en Hierarchy.
2 WallLeft	Dentro de Walls cree un GameObject vacío llamado WallLeft. Add Component → BoxCollider2D. En el componente, establezca Size (0.1, 12) y en Transform Position (−9, 0, 0). Asigne Layer = Wall.
3 WallRight	Repita el proceso con WallRight: BoxCollider2D Size (0.1, 12), Position (9, 0, 0), Layer = Wall.
4 WallTop	Cree WallTop: BoxCollider2D Size (18, 0.1), Position (0, 5.5, 0), Layer = Wall.
5 Sin pared inferior	No cree una pared en la parte inferior. La pelota debe poder caer para activar la condición de derrota.

Paso 6: Crear el bloque destructible y su Prefab

1 Crear objeto	GameObject → 2D Object → Sprites → Square. Renombre a Brick. Transform: Scale (1.5, 0.5, 1). Asigne un color desde SpriteRenderer.
2 BoxCollider2D	Add Component → Box Collider 2D. No agregar Rigidbody2D; el bloque es estático.
3 Script Brick	En Project cree una carpeta Scripts y cree un MonoBehaviour Brick. Abra el script y reemplace el contenido con el código de la sección 5.3. Asigne el script a Brick. Asigne Layer = Brick.
4 Crear Prefab	En Project cree la carpeta Assets/Prefabs. Arrastre el GameObject Brick desde Hierarchy hacia esa carpeta. Se crea el Prefab. Elimine la instancia de la escena (clic derecho → Delete).

Paso 7: Script BrickManager — cuadrícula de bloques

1 Objeto vacío	Cree un GameObject vacío llamado BrickManager en Hierarchy.
2 Script	Cree un MonoBehaviour BrickManager. Copie el código de la sección 5.6. Y asígnelo al objeto BrickManager.
3 Configurar	En el Inspector de BrickManager: arrastre el Prefab Brick al campo Brick Prefab. Configure Rows = 4, Columns = 8 (o según diseño).
4 Probar	Presione Play. Deben aparecer filas de bloques en la parte superior de la pantalla con los colores definidos en el Inspector.

Paso 8: Scripts de movimiento — PaddleController y BallController

1 PaddleController	Cree un MonoBehaviour PaddleController. Copie el código de la sección 5.1. Y asígnelo al objeto Paddle.
2 BallController	Cree un MonoBehaviour BallController. Copie el código de la sección 5.2. Y asígnelo al objeto Ball.
3 Tag pelota	Verifique que el GameObject Ball tenga asignado el Tag "Ball". Los scripts de Brick y DeathZone usan CompareTag("Ball") para identificarla.
4 Probar	Presione Play. La paleta debe moverse con A/D o las flechas. Al presionar Espacio la pelota se lanza. Verifique que rebota en las paredes y en la paleta.

Paso 9: Sistema de puntuación con UI

1 Canvas	GameObject → UI → Canvas. En el Inspector del Canvas, configure UI Scale Mode → Scale With Screen Size con Reference Resolution 1920×1080.
2 Score Text	Dentro del Canvas: GameObject → UI → Text - TextMeshPro → renombre a ScoreText. En RectTransform: ancla Top-Left, posición (20, −20). Cambie el color del texto a blanco.
3 Lives Text	Repita el proceso con un segundo texto TMP llamado LivesText. Ubíquelo en la esquina superior derecha.
4 GameManager	Cree un GameObject vacío llamado GameManager. Cree un MonoBehaviour GameManager. Copie el código de la sección 5.4. Y asígnelo al objeto GameManager. Arrastre ScoreText y LivesText a los campos correspondientes en el Inspector.

Paso 10: Zona de muerte (DeathZone) — condición de derrota

1 Objeto vacío	Cree un GameObject vacío llamado DeathZone.

2 BoxCollider2D	Add Component → Box Collider 2D. Size (20, 0.5), Position (0, −6, 0). Active IsTrigger = true en el Inspector del collider.
3 Script	Cree un MonoBehaviour DeathZone. Copie el código de la sección 5.5. Y asígnelo al objeto DeathZone.
4 ¿Por qué Trigger?	La zona de muerte usa Trigger (no Collision) porque no debe bloquear ni rebotar la pelota, solo detectar que cruzó el límite inferior. Si usara Collision, la pelota rebotaría invisible bajo la pantalla.

Paso 11: Configurar la Layer Collision Matrix

1 Abrir ajustes	Edit → Project Settings → Physics 2D. Desplace hasta encontrar la sección Layer Collision Matrix.
2 Desactivar	Desactive las siguientes interacciones (clic en la casilla para quitarla): Paddle ↔ Brick, Paddle ↔ Wall, Brick ↔ Brick, Brick ↔ Wall, Ball ↔ Ball.
3 Mantener	Deje activas: Ball ↔ Paddle, Ball ↔ Brick, Ball ↔ Wall. Estas son las únicas colisiones necesarias para el juego.
4 Prueba final	Presione Play y compruebe el juego completo: la pelota rebota, destruye bloques, el puntaje se actualiza y al caer debajo se activa el sistema de vidas.

5. Código Completo Comentado

5.1 PaddleController.cs

La paleta usa Kinematic Rigidbody2D, por lo que su movimiento se realiza mediante MovePosition en lugar de modificar transform.position directamente. Esto garantiza compatibilidad con el sistema de física y activa correctamente OnCollisionEnter2D cuando la pelota la toca.

```
using UnityEngine;

// Controlador de la paleta (Paddle)
// Requiere: Rigidbody2D con Body Type = Kinematic
public class PaddleController : MonoBehaviour
{
    [Header("Movimiento")]
    public float speed = 10f;        // Velocidad horizontal
    public float boundaryX = 7.5f;   // Límite de pantalla

    private Rigidbody2D rb;

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    // FixedUpdate: ideal para física, se ejecuta a frecuencia fija (50 Hz por
defecto)
    void FixedUpdate()
    {
        float input = Input.GetAxis("Horizontal"); // A/D o flechas: valor -1 a 1

        float newX = rb.position.x + input * speed * Time.fixedDeltaTime;
        newX = Mathf.Clamp(newX, -boundaryX, boundaryX);

        // MovePosition respeta la física Kinematic
        rb.MovePosition(new Vector2(newX, rb.position.y));
    }
}
```

5.2 BallController.cs

Para mantener la velocidad constante después de cada rebote, el script normaliza el vector de velocidad en cada FixedUpdate. Esto corrige pequeñas variaciones acumuladas por el motor de física.

```
using UnityEngine;

// Controlador de la pelota
// Requiere: Rigidbody2D (Dynamic) + CircleCollider2D + BallMaterial (bounce=1,
friction=0)
public class BallController : MonoBehaviour
{
    [Header("Velocidad")]
```

```
    public float startSpeed = 7f;

    private Rigidbody2D rb;
    private bool isLaunched = false;

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    void Update()
    {
        if (!isLaunched && Input.GetKeyDown(KeyCode.Space))
            Launch();
    }

    void FixedUpdate()
    {
        // Mantener velocidad constante: normalizar y reescalar
        if (isLaunched && rb.linearVelocity.magnitude > 0.01f)
            rb.linearVelocity = rb.linearVelocity.normalized * startSpeed;
    }

    void Launch()
    {
        isLaunched = true;
        // Dirección inicial diagonal hacia arriba-derecha (45°)
        Vector2 dir = new Vector2(0.5f, 1f).normalized;
        rb.linearVelocity = dir * startSpeed;
    }

    // Llamado por GameManager al perder una vida
    public void ResetBall(Vector2 startPos)
    {
        isLaunched = false;
        rb.linearVelocity = Vector2.zero;
        transform.position = startPos;
    }
}
```

5.3 Brick.cs

Cada bloque detecta la colisión física de la pelota mediante OnCollisionEnter2D (no Trigger), notifica al GameManager los puntos ganados y se destruye. Si hitPoints es mayor a 1, el bloque cambia de color al recibir impactos.

```
using UnityEngine;

// Script de bloque destructible
// Requiere: BoxCollider2D (IsTrigger = false)
public class Brick : MonoBehaviour
{
    [Header("Propiedades")]
    public int pointValue = 10;   // Puntos al destruirse
    public int hitPoints  = 1;    // Golpes necesarios

    private SpriteRenderer sr;
```

```
    void Start()
    {
        sr = GetComponent<SpriteRenderer>();
    }

    // OnCollisionEnter2D: colisión FÍSICA (IsTrigger = false)
    // El parámetro Collision2D contiene datos del contacto
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Ball"))
            TakeHit();
    }

    void TakeHit()
    {
        hitPoints--;
        if (hitPoints <= 0)
        {
            GameManager.Instance.AddScore(pointValue);
            Destroy(gameObject);
        }
        else
        {
            // Feedback visual: oscurecer el color
            sr.color = Color.Lerp(sr.color, Color.gray, 0.5f);
        }
    }
}
```

5.4 GameManager.cs

El GameManager implementa el patrón Singleton: una instancia global y estática accesible desde cualquier script con GameManager.Instance. Administra el puntaje, las vidas y las condiciones de fin de juego.

```
using UnityEngine;
using TMPro;
// Gestor del juego — patrón Singleton
public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    [Header("UI")]
    public TMP_Text scoreText;
    public TMP_Text livesText;
    public GameObject gameOverPanel;

    [Header("Configuración")]
    public int startingLives = 3;

    private int score = 0;
    private int lives;

    void Awake()
    {
        if (Instance != null && Instance != this) { Destroy(gameObject); return; }
```

```
        Instance = this;
    }

    void Start()
    {
        lives = startingLives;
        UpdateUI();
        if (gameOverPanel != null) gameOverPanel.SetActive(false);
    }

    public void AddScore(int points)
    {
        score += points;
        UpdateUI();
    }

    public void LoseLife()
    {
        lives--;
        UpdateUI();
        if (lives <= 0)
            GameOver();
        else
        {
            BallController ball = FindFirstObjectByType<BallController>();
            if (ball != null) ball.ResetBall(new Vector2(0f, -3f));
        }
    }

    void GameOver()
    {
        Time.timeScale = 0f;
        if (gameOverPanel != null) gameOverPanel.SetActive(true);
    }

    void UpdateUI()
    {
        if (scoreText != null) scoreText.text = "Puntaje: " + score;
        if (livesText  != null) livesText.text  = "Vidas: "   + lives;
    }
}
```

5.5 DeathZone.cs

La zona de muerte usa OnTriggerEnter2D porque no queremos que bloquee físicamente la pelota, solo detectar que cruzó el límite inferior. Si se usara Collision, la pelota rebotaría de forma invisible debajo de la pantalla.

```
using UnityEngine;

// Zona de muerte — detecta cuando la pelota cae bajo la paleta
// Requiere: BoxCollider2D con IsTrigger = true
public class DeathZone : MonoBehaviour
{
    // OnTriggerEnter2D: los objetos se ATRAVIESAN, solo se detecta el overlap
    // El parámetro es Collider2D (no Collision2D)
    private void OnTriggerEnter2D(Collider2D other)
    {
```

```
        if (other.CompareTag("Ball"))
            GameManager.Instance.LoseLife();
    }
}
```

5.6 BrickManager.cs

Genera la cuadrícula de bloques en tiempo de ejecución (en Start). Permite cambiar el número de filas, columnas y colores directamente desde el Inspector sin modificar el código.

```
using UnityEngine;

public class BrickManager : MonoBehaviour
{
    [Header("Prefab")]
    public GameObject brickPrefab;

    [Header("Cuadrícula")]
    public int rows    = 4;
    public int columns = 8;
    public float brickWidth  = 1.6f;
    public float brickHeight = 0.6f;
    public Vector2 startPosition = new Vector2(-5.6f, 3f);

    [Header("Colores por fila")]
    public Color[] rowColors = {
        new Color(0.28f, 0f,   0.51f),   // Indigo oscuro
        new Color(0.20f, 0.4f, 1f),      // Azul
        new Color(0f,    0.7f, 0.7f),    // Teal
        new Color(0.1f,  0.7f, 0.3f),   // Verde
    };

    void Start() { GenerateGrid(); }

    void GenerateGrid()
    {
        for (int row = 0; row < rows; row++)
        {
            for (int col = 0; col < columns; col++)
            {
                float x = startPosition.x + col * brickWidth;
                float y = startPosition.y - row * brickHeight;

                GameObject brick = Instantiate(brickPrefab,
                    new Vector2(x, y), Quaternion.identity);

                SpriteRenderer sr = brick.GetComponent<SpriteRenderer>();
                if (sr != null && row < rowColors.Length)
                    sr.color = rowColors[row];
            }
        }
    }
}
```

Explicación del código

- OnCollisionEnter2D (Brick.cs): se activa cuando hay contacto físico real entre dos Collider2D que no son Trigger. Recibe un parámetro Collision2D con datos del impacto (puntos de contacto, normal, velocidad relativa).
- OnTriggerEnter2D (DeathZone.cs): se activa cuando un Collider2D con IsTrigger = true se solapa con otro collider. Recibe solo el Collider2D del objeto que entró, sin datos de física.
- GameManager.Instance: el patrón Singleton asegura que exista un único GameManager accesible globalmente. Se instancia en Awake() y cualquier script puede llamar a GameManager.Instance.AddScore() o .LoseLife() sin necesidad de buscar referencias.
- rb.velocity.normalized * startSpeed: normalizar el vector velocidad y reescalarlo garantiza que la pelota siempre mantenga la misma rapidez, independientemente de la dirección del rebote.
- Time.fixedDeltaTime: equivalente a deltaTime pero para FixedUpdate. Garantiza movimiento independiente de la tasa de actualización del motor de física.

6. Ejercicios Propuestos

Realice las siguientes variaciones del mini-juego para reforzar los conceptos aprendidos. Está diseñado para que complete al menos dos de los cuatro ejercicios.

Ejercicio 1: Pelota que acelera (Nivel básico)

Modifique BallController.cs para que la velocidad de la pelota aumente en un 5% cada vez que destruye un bloque. Implemente una velocidad máxima (maxSpeed = 15f) para que el juego no se vuelva injugable.

- Agregue una variable public float maxSpeed = 15f al script.
- Cree un método público IncreaseSpeed(float factor) que multiplique startSpeed por (1 + factor) con un límite de maxSpeed.
- En Brick.cs, antes de Destroy(gameObject), llame a FindObjectOfType<BallController>().IncreaseSpeed(0.05f).

Ejercicio 2: Bloques con múltiples puntos de vida (Nivel intermedio)

Diseñe tres tipos de bloques: Normal (hitPoints = 1, 10 pts), Resistente (hitPoints = 2, 25 pts) e Indestructible (visual diferente, no se destruye). Cree un Prefab por tipo y modifique BrickManager para generar una cuadrícula con tipos mixtos.

- Para el bloque Indestructible, use hitPoints = −1 y en TakeHit() verifique if (hitPoints == −1) return; antes de decrementar.
- Cambie el color del SpriteRenderer en Start() según el tipo, usando un Color basado en hitPoints.
- En BrickManager, genere el tipo de bloque según un array de configuración por fila.

Ejercicio 3: Sistema de Power-Ups (Nivel avanzado)

Al destruir ciertos bloques (con probabilidad aleatoria), instanciar un power-up que cae con gravedad. Si la paleta lo recoge mediante un Trigger, activar un efecto temporizado. Implementar al menos dos efectos: Paleta Grande (duración 10 s) y Bola Extra.

- El power-up usa Rigidbody2D Dynamic con Gravity Scale = 1 para caer naturalmente.
- La paleta tiene un segundo BoxCollider2D hijo con IsTrigger = true para recoger power-ups sin afectar el rebote de la pelota.
- Use Coroutines (IEnumerator + StartCoroutine) para los efectos con duración temporal.
- En Brick.cs genere el power-up con: if (Random.Range(0f, 1f) < 0.25f) Instantiate(powerUpPrefab, ...).

Ejercicio 4: Pantalla de Game Over y reinicio (Nivel básico)

Cree un Canvas con un panel de Game Over que muestre el puntaje final y un botón de reinicio. Al pulsarlo, restablecer la escena con SceneManager.

- Agregue using UnityEngine.SceneManagement; al script.
- El botón llama a: Time.timeScale = 1f; SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
- El panel se activa desde GameManager.GameOver() con gameOverPanel.SetActive(true).

- En Project Settings → Player → Build, asegúrese de que la escena esté en la lista Build Scenes.

8. Bibliografía y Recursos Adicionales

Bibliografía

Hocking, J. (2018). Unity in action: Multiplatform game development in C# (2nd ed.). Manning Publications.

Ferrone, H. (2023). Learning C# by Developing Games with Unity (8th ed.). Packt Publishing.

Gregory, J. (2019). Game engine architecture (3rd ed.). CRC Press.

Unity Technologies. (2024). Rigidbody 2D. Unity Manual. https://docs.unity3d.com/Manual/class-Rigidbody2D.html

Unity Technologies. (2024). Physics Material 2D. Unity Manual. https://docs.unity3d.com/Manual/class-PhysicsMaterial2D.html

Unity Technologies. (2024). Collider 2D. Unity Manual. https://docs.unity3d.com/Manual/Colliders2D.html

Unity Technologies. (2024). Layer-based collision detection. https://docs.unity3d.com/Manual/LayerBasedCollision.html

Recursos en línea

- Documentación oficial de Unity: https://docs.unity3d.com
- Unity Learn (tutoriales gratuitos): https://learn.unity.com
- Referencia de la API de Unity (C#): https://docs.unity3d.com/ScriptReference/
- Canal de YouTube oficial de Unity: https://www.youtube.com/@unity